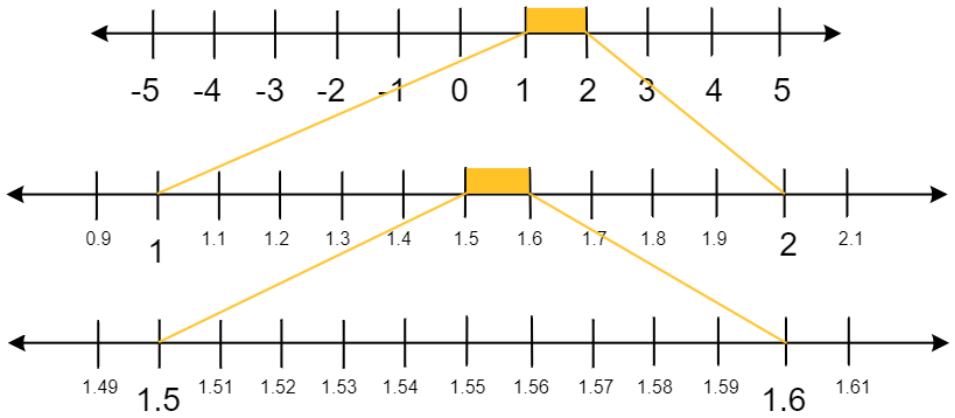
CS402 – High-Performance Computing

Fundamental Ideas – Representing Numbers

Gihan Mudalige
Professor
Department of Computer Science
University of Warwick
g.mudalige@warwick.ac.uk

Continuous Mathematics on a Discrete System

- Numbers from $\mathbb{R}$ **(set of all real numbers)** cannot be handled directly by computers
  - Numbers in $\mathbb{R}$ have **infinitely many digits** – have to truncate to fit into a finite number of bits
  - So how do we go about representing numbers on computers ?



- Computers needs to use a fixed number of bytes per number to store values from $\mathbb{R}$
  - Machines could work with very flexible formats – bits used per number could be arbitrarily large
  - However, this will lead to an explosion of your memory footprint !
  - Hardware would be very slow
  - What to do about "unstorable" numbers : e.g. 1/3, π

Only option - use finite number of bits and approximate "the real thing".

OUTLINE

- Fixed-point representation
- Error, Accuracy and Precision, Truncation and Rounding
- Floating-point representation and IEEE standard
- Machine Precision
- Comparing Numbers
- Reproducibility

FIXED-POINT NUMBERS

- Write the number with a certain number of digits before and after the decimal (binary) point

$$3.625|_{10} \rightarrow \quad 11.101|_2 \rightarrow \quad 0011.1010 \text{ (in 8 bits)}$$
$$6.5|_{10} \quad \rightarrow \quad 110.1|_2 \quad \rightarrow \quad 0110.1000 \text{ (in 8 bits)}$$

- Fixed-point is useful in some circumstances e.g. financial calculation

- However, there is significant limitations especially for scientific computation
  - Only suited to hold numbers from a limited range: e.g 8-bits: 0000.0000 to 1111.1111 = $0|_{10}$ to $15.9375|_{10}$
  - Would require an increasingly large number of bits to represent a larger and larger range

Mass of sun in grams : 1989 000 000 000 000 000 000 000 000 000 000
Mass of an electron in grams: 0.000 000 000 000 000 000 000 000 000 910 938

Roughly 14 Bytes + 12 Bytes = 26 Bytes (208 bit) storage would be needed to represent both these numbers

Observation - Large number of bits will be holding zeros !

Relative and Absolute Error

- Consider a fixed-point format with 2 decimal digits before and after decimal point
  - Then to represent $3|_{10}$ we can use : 03.00
  - To represent $\frac{1}{3}|_{10}$ we can only do : 00.33 → Result is incorrect by 0.003333…

- Given the finite number of bits to represent a number in a machine we have to cut the bits of the real number off introducing an error

- If x is the real number and x_m is its machine representation then,

  - Absolute Error:

$$e = |x_m - x|$$

  - Relative Error:

$$\epsilon = \frac{\mathrm{e}}{|x|} = \left|\frac{x_m - x}{x}\right|$$

- For above example : absolute error = 0.003333… and relative error = 0.01 = 1%

Range, Precision, Accuracy

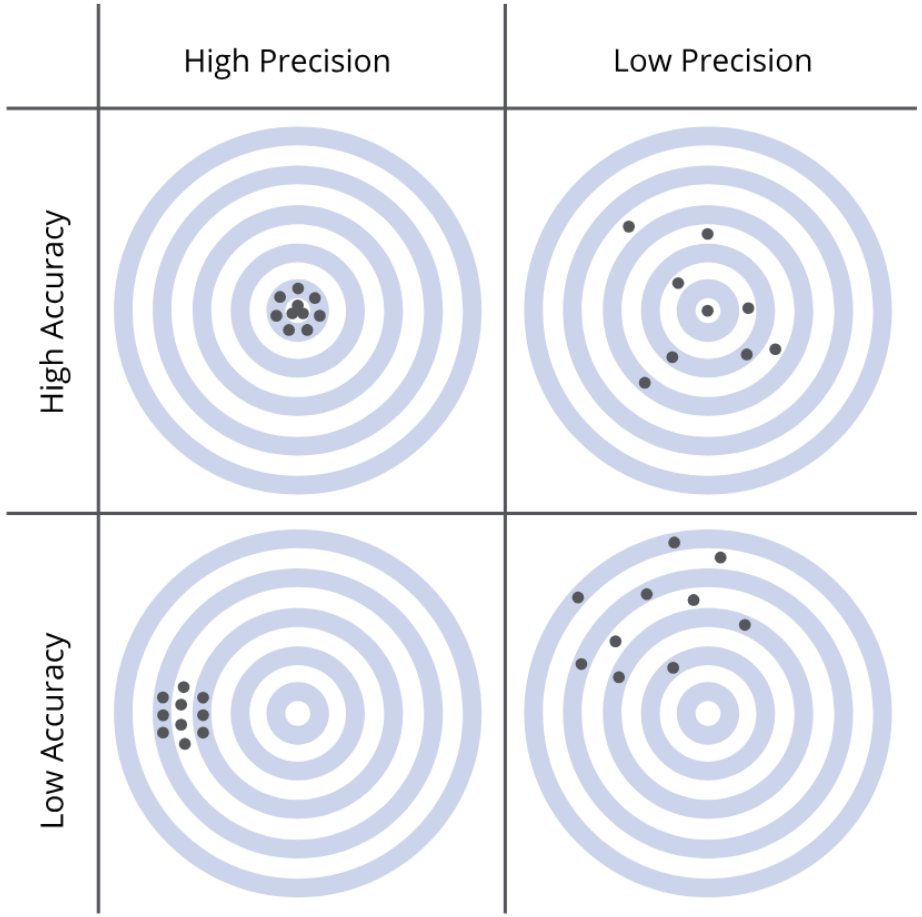
- **Range** – how big or how small a number can be

- **Precision** – measure of the exactness of a number
  - Corresponds to the number of significant digits used
  - real number $x \in R$ in mathematics has an infinite precision

- **Accuracy** – measure of the correctness of a quantity

- Example consider Pi = 3.141 and Pi = 3.241592
  - 3.141 is a low precision number but is more accurate than 3.241592 which has an error in the 2nd digit

- Need to design algorithms that **preserves the accuracy** that the **available precision allows**



https://www.ossila.com/pages/accuracy-vs-precision

Floating Point Numbers

- Need to cover a larger range of values, but format should run as efficient as possible on hardware
  - memory footprint should be under control → can fit into a fixed number of bytes

- **Floating-point** - Scheme where the number of positions after the decimal point is not fixed
- The decimal (or binary) point is not fixed in a position but can "float" around

$$1234.56|_{10} \rightarrow 0.123456 \times 10^4|_{10}$$
$$1001101.0010100|_2 \rightarrow 0.10011010010100 \times 2^{111}|_2$$

$$\boxed{[\text{mantissa}] \times \text{radix}^{[\text{exponent}]}}$$

- Both mantissa (also called significand) and exponent stored using the fixed number of bytes available

  Assume 8-bit representation – 5 bits for mantissa and 3 bits for exponent

$$1001101.0010100|_2 \xrightarrow{\text{normalize}} 0.10011010010100 \times 2^{111}|_2 \xrightarrow{\text{truncate / round}} 0.10011 \times 2^{111}|_2 \rightarrow \boxed{111 \mid 10011}$$

Range of 8-bit representation: $0.10000|_2 \times 2^0 = \frac{1}{2}|_{10} = 0.5$ to $0.11111|_2 \times 2^7 = 124$

Significantly larger range than fixed point

Truncation vs Rounding

$$1001101.0010100|_2 \xrightarrow{\text{normalize}} 0.10011010010100 \times 2^{111}|_2 \xrightarrow{\text{truncate / round}} 0.10011 \times 2^{111}|_2 \rightarrow \boxed{111 \mid 10011}$$

- **Truncating** - Dropping/chopping off bits (to fit get number to fit into the available number of bits)
  - Truncating creates an error called *induced* error – this is a *biased* error as the number after truncation is always smaller than the original

- **Rounding** – Instead of dropping/chopping off bits *round* the number to fit
  - If value of lost digits > ½(LSB of the retained digits) → add 1 to LSB of the retained digits
    Else → add 0 to LSB of the retained digits

E.g. 0.1101101 → Truncate to 4 significant bits → 0.1101
0.1101101 → Rounding to 4 significant bits (removing 101) → 0.1101 + 1 → 0.1110

- Rounding always preferred over truncation
  - Gives rise to an *unbiased* error whereas truncation leads to a systematic error
  - Disadvantage is that rounding requires a further arithmetic operation to be performed on the result

IEEE – 754 Floating-point Representation

- Normalized Numbers in IEEE floating-point have the form

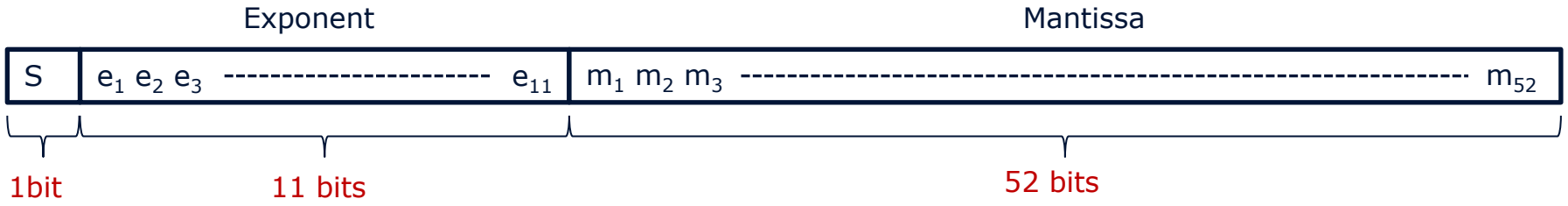
$$[\pm]1.[\textbf{mantissa}] \times 2^{[\textbf{exponent}]}$$

signed bit — implied — implied

- IEEE number of bits for different precision

Precision	Sign	Exponent	Mantissa	Total
Single (FP32)	1	8	23	32
Double (FP64)	1	11	52	64
Extended	1	15	64	80
Quad (FP128)	1	15	112	128

- Double (FP64)



IEEE Floating-point Representation - Exponent

- Exponent - 11 bits = 2^{11} = 2048 different integers
- 2 integers represent special cases (talk about this later), then can represent 2046 different integers

- Need to represent both positive and negative numbers – Introduce a bias
  - Biased exponent – store **exponent**+127 (single precision) or **exponent**+1023 (double precision)
  - Then value stored is then always positive – no need for an explicit sign bit
  - Range : -126 <= exponent <= 127 (single precision) or -1022 <= exponent <= 1023 (double precision)

- Example numbers :
  - $-1010011.101101011|_2 \rightarrow$

1	10000000101	0100111011010110000000000000000000000000000000000000000

Exp.=6+1023
=$1029|_{10}$

  - $11010.01101\ |_2 \rightarrow$

0	10000000011	1010011010000000000000000000000000000000000000000000000

Exp.=4+1023
=$1027|_{10}$

IEEE Floating-point Representation – Special Cases

- **Exponent all 1s** (i.e. $2047|_{10}$) **+ mantissa non-zero** : NaN (not a number)

1	11111111111	1100000000000000000000000000000000000000000000000000

  - Allows to represent undefined or invalid floating-point results (0/0, 0×∞ etc.)
  - Without NaNs hardware will be force to return a fake number (incorrect) or trap/crash for every occurrence
  - NaN can propagate through calculations : Nan + x = NaN
  - This contaminates downstream results – but far safer than silently returning zero or some arbitrary value

- **Exponent all 1s** (i.e. $2047|_{10}$) **+ mantissa all 0s** (or 100…0, where 1 is implicit): ±∞ (infinity)

±	11111111111	0000000000000000000000000000000000000000000000000000

- **Exponent all 0s + mantissa all 0s** (no implicit 1 assumed): ±0 (zero)

±	00000000000	0000000000000000000000000000000000000000000000000000

IEEE Floating-point Representation – Special Cases : Denormalized Numbers

- **Exponent all 0s** (i.e. $0|_{10}$) **+ mantissa non-zero** (no implicit 1 assumed): denormalized or subnormal numbers

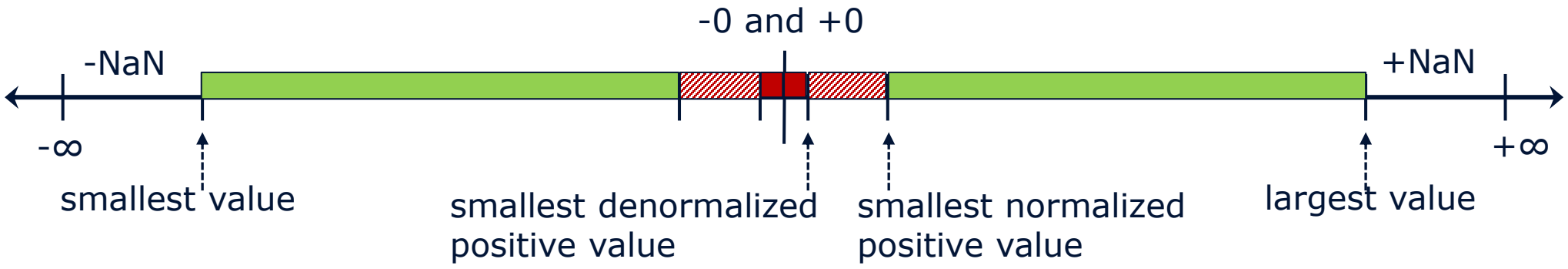
±	00000000000	----------non zero----------

  - Allow gradual underflow, so very small values don't suddenly become zero

- However, should be avoided in practice
  - Indicates values are close to the **machine precision** limit
  - Small perturbations (rounding, noise) are as large as the value itself
  - It's a safety net, signals that - the computation is no longer numerically trustworthy, even if it hasn't yet become zero

- Also much slower to process
  - Most FP hardware optimized for normalized numbers (e.g. assumes implicit 1 etc.)
  - Denormals need extra steps are needed to align and normalize operands
  - performance-critical code often avoid them or flush them to zero

IEEE Floating-point Representation : Regions on the number line

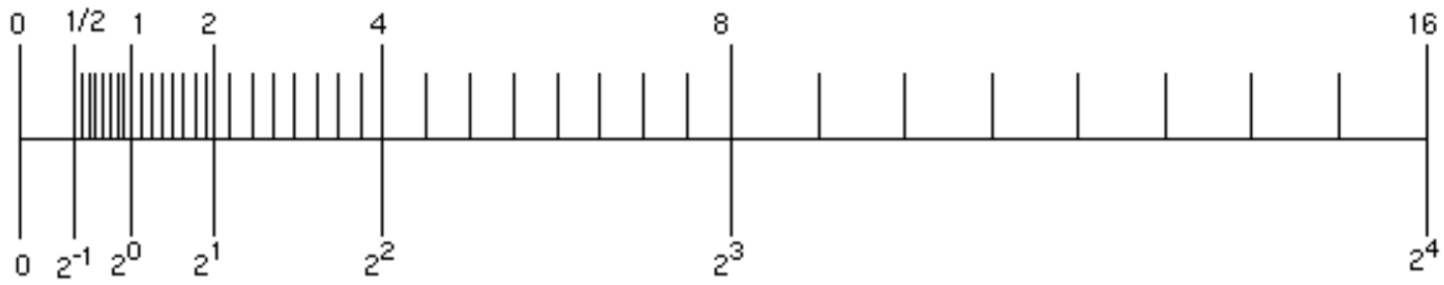


IEEE Floating-point Representation

- We know that FP numbers are stored as :

$$\text{value} = [\text{mantissa}] \times 2^{[\text{exponent}]}$$

  - the mantissa (also called significand) has a **fixed number of bits**
  - when exponent increased – each step between adjacent numbers gets larger
  - <u>spacing</u> roughly **doubles** each time the exponent increases by 1 (exactly double at boundaries)

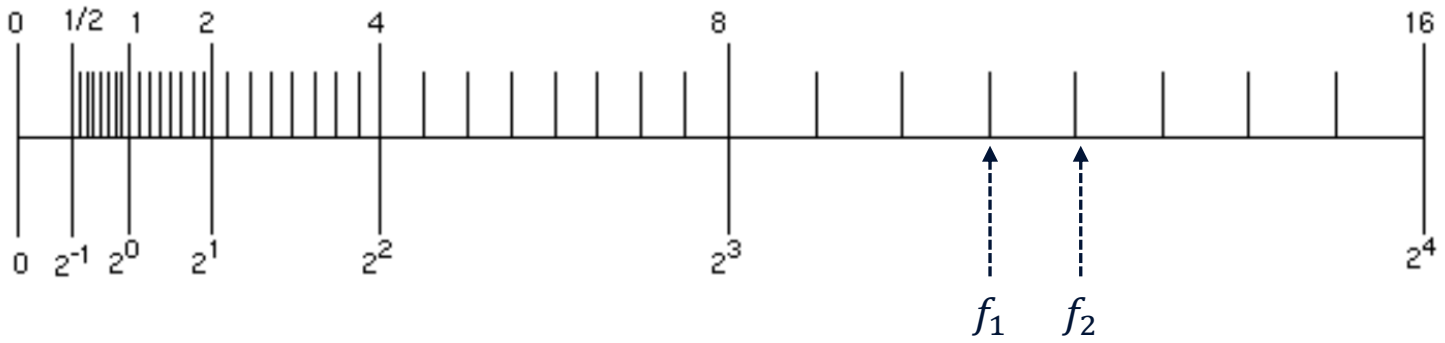


- Each power-of-two interval has the **same number of representable values**, but stretched over a larger range

Floating-point numbers have constant *relative* precision, <u>not</u> constant *absolute* precision.

Machine Precision

- **Machine precision (ϵ)** is the smallest *relative* spacing between adjacent representable FP numbers



$$\text{Machine precision} = \epsilon = \frac{f_2 - f_1}{f_1}$$

- Another definition - **Machine precision (ϵ)** is the **smallest difference between two distinct floating-point numbers around 1.0** that the computer can represent.
  - Machine precision is the smallest positive FP number such that: $1 + \epsilon \neq 1$

The machine precision is the difference between 1.0 and the next number closest to 1.0 that we can represent in our floating-point format

Find Machine Precision

- Start with a high estimate of ϵ and successfully reduce it (by dividing by 2) until the code cannot distinguish any difference between 1 and $1+\epsilon$

```
double epsilon = 1.0;
double previousGuessOfEpsilon = 1.0;

while((1.0 + epsilon) != 1.0){
        previousGuessOfEpsilon = epsilon;
        epsilon = epsilon / 2.0;
}
```

- For IEEE SP representation (FP32) : $1+\epsilon = +1.000.....001|_2 = 1+2^{-23}$

  $\epsilon = 2^{-23} \approx 10^{-7}$

- For IEEE DP representation (FP64) : $1+\epsilon = +1.000.....001|_2 = 1+2^{-52}$

  $\epsilon = 2^{-52} \approx 10^{-16}$

Rounding Error

- Representing real numbers using an FP representation in a computer requires **rounding** the actual number to fit one of the representable FP numbers → this introduces an error

$$0.1|_{10} = 0.0001100110011001100110011001\ldots|_2$$

Normalize: $= 1.1001100110011001100110011001\ldots|_2 \times 2^{-4}$ [note 1001 repeats]

Round: $(0.1)_M = 1.\underbrace{1001100110011001100110011001100110011001100110011001100110\mathbf{10}}_{52\text{ bits}}|_2 \times 2^{-4}$

Truncated tail amount is $= 0.\overline{1001}|_2 \times 2^{-52} \times 2^{-4} \quad = 0.6 \times 2^{-56}|_{10}$

Round up $=$ add $1\times 2^{-52}\times 2^{-4} \quad = 1.0 \times 2^{-56}|_{10}$

$(0.1)_M = 0.1 + 0.4 \times 2^{-56}|_{10}$

Therefore absolute error $= \text{abs}(\ 1.0 \times 2^{-56} - 0.6 \times 2^{-56}\ ) = 0.4 \times 2^{-56}$

Then relative error $= 0.4 \times 2^{-56} / 0.1 \quad = 4 \times 2^{-56} = 2^{-54} \approx 5.55\times 10^{-17}$ $\left(\leq \frac{2^{-52}}{2}\right.$ i.e $\epsilon/2)$

$$\boxed{\text{Relative error of rounding is always} \leq \epsilon/2}$$

FP Numbers in Action

- Adding two numbers with different exponents :

$$x = 1.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ |_2 \times 2^{10}$$

$$y = 1.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ |_2 \times 2^{-44}$$

Align exponents and add: (CPU will do the addition in higher number of bits, e.g. 80-bits)

53rd bit

$$x+y = (1.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 00\ +$$

$$0.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 01\ ) \times 2^{10}$$

$$\text{Exact sum} = 1.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 01\ \times 2^{10}$$

Rounding:

$$(x+y)_M = 1.\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001$$

$$(x+y)_M = x$$

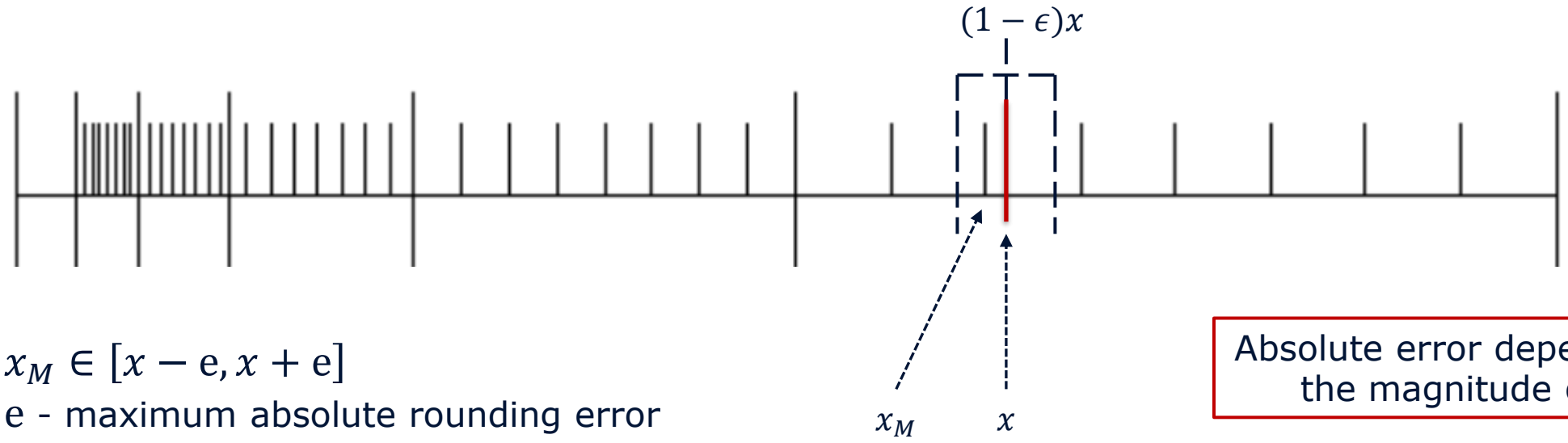
Absolute Error : $|y| = 2^{-44}$

Relative Error : $|y|/|x+y| \approx |y|/|x| = 2^{-44} / 2^{10} = 2^{-54} \approx 5.55 \times 10^{-17}$

FP Numbers in Action

- After any operation—besides some special cases like the multiplication with two — the system will apply round, and thus "spoil" the result

Therefore From the very beginning we are computing on slightly incorrect values



$x_M \in [x - \mathrm{e}, x + \mathrm{e}]$

e - maximum absolute rounding error

Absolute error depends on the magnitude of x

- If x_M is the machine representation of number and $\epsilon = \pm e/x$ i.e relative error, then:

$$x_M = x(1+\epsilon)$$

Relative error bounded by machine precision

Inexact Arithmetic

- When working with FP numbers our most basic operations are "wrong"

$$f = x + y \quad \rightarrow \quad f_M = \mathrm{round}(x + y) = (x + y)_M$$

Notation: $$(x + y)_M = x +_M y = (x + y)(1 + \epsilon)$$

- ϵ is the **worst-case estimate** of the actual rounding error – read it as "± the machine precision"
  - It can be positive, negative or zero
  - Its value unknown, but bounded where actual $\bar{\epsilon}$ value: $|\bar{\epsilon}| \leq \epsilon$

- Assuming worst case rounding error, for any operation we can write :

$$\begin{aligned} x +_M y &= (x + y)(1 + \epsilon) \\ x -_M y &= (x - y)(1 + \epsilon) \\ x \times_M y &= (x \times y)(1 + \epsilon) \\ x /_M y &= (x \ / \ y)(1 + \epsilon) \end{aligned}$$

- Can further simplify : $\epsilon \times \epsilon = 0$ (higher order terms discarded)
- Additionally introduce : $\epsilon + \epsilon = 2e$ and $\epsilon - \epsilon = 2e$ (worst case given sign not known)

FP Arithmetic is not associative

Floating-point arithmetic is <u>not associative:</u> each operation is rounded independently, changing the order of evaluation changes where rounding errors occur and therefore changes the final result.

- Case 1: $(x + y) + z$

$$\left((x+y)_M + z\right)_M = \left((x+y)(1+\epsilon_1) + z\right)(1+\epsilon_2)$$

$$= x + y + z + (x+y)\epsilon_1 + (x+y+z)\epsilon_2$$

Error terms depends on $(x + y)$

E.g. x = $1e^{16}$, y = $-1e^{16}$, z = 1

$(x + y) = 1e^{16} - 1e^{16} = 0$

$(x + y) + z = 0 + 1 = 1$

- Case 2: $x + (y + z)$

$$(x + (y+z)_M)_M = \left(x + (y+z)(1+\epsilon'_1)\right)(1+\epsilon'_2)$$

$$= x + y + z + (y+z)\epsilon'_1 + (x+y+z)\epsilon'_2$$

Error terms depends on $(y + z)$

E.g. x = $1e^{16}$, y = $-1e^{16}$, z = 1

$(y + z) = -1e^{16} + 1 = -1e^{16}$

$x + (y + z) = 1e^{16} + (-1e^{16}) = 0$

Compiler Optimizations and FP operations

- Compilers (especially when optimizing) apply transformation such as
  - Re-associating expressions, regrouping additions, vectorising
  - Unless told otherwise compiler assumes : **"The programmer doesn't care about exact FP rounding."**

- Default Compile behavior (`gcc -O0, -O3`)
  - Do not re-associate FPG expressions, preserve written order
  - Respect IEEE-754 semantics
  - Safe, slower but predictable calculations

- Fast-math falgs (`gcc -Ofast | gcc -ffast-math | clang -ffast-math`)
  - reschedule additions, reorder reductions
  - drop NaN/Inf checks
  - assumes no overflow

- Can make **absolutely explicit** to not relax IEEE FP : `-O3` + strict FP flags e.g `-fno-fast-math`
  - Real projects might relax FP elsewhere, so can be explicit to force IEEE FP
  - Especially important on GPUs, **check default behavior**

EXAMPLE SUMMATION LOOP

- Consider simple C code to sum numbers :

```
double summation(double *u) {
  double sum = 0.0;
  for(int i=0; i<1000000; i++) {
    sum = sum + u[i];
  }
  return sum;
}
```

- Compiled **without optimization**
  - Historically adds the numbers in 80-bit precision - e.g on 32-bit, use x87 FPU, keeps **sum** in a x87 register
  - Rounding to 64-bit only when the value is stored to memory
  - More precision, but slow !

- Compiled **with optimization** (`-O2, -O3`)
  - Compiler switches to SSE/AVX instructions, vectorises the loop (SSE/AVX does not support 80-bit precision)
  - All arithmetic in **64-bit IEEE-754 double precision**
  - **Parallel sums are rounded at every step**
  - Fast, but less precision

Example summation loop – Parallel reduction : OpenMP and MPI

- Add OpenMP Pragma:

```
double summation(double *u) {
  double sum = 0.0;
  #pragma omp parallel for shared(u) reduction(+:sum)
  for(int i=0; i<1000000; i++) {
    sum = sum + u[i];
  }
  return sum;
}
```

- Using 10 threads:
  - break the sum up into 10 sections of 100000 elements (decomposition can vary dynamically !)
  - sum each of those – perhaps vectorize each and then combine them at the end
  - **Changes result every time you run this**

- Distributed-memory MPI:
  - parts of the sums on different computers, then combines the results
  - Even with the same number of processors, the order in which the partial sums are combined can vary, depending on which gets finished first
  - **almost always produce different results per run**

Getting bit-wise identical results is very difficult and most HPC work consider it unnecessary.

Programming Guidelines for FP numbers

Never compare FP numbers with the == operator !

- In C the operator == does a left-to-right bitwise comparison

- As we work with imprecise numbers subject to round-offs, need to write code that checks if *reasonably close*

```
if(std::abs(x-y)/std::max(std::abs(x),std::abs(y)) < 1e-10) {
  // x and y are equal in floating point precision
}
```

Computer will not check for NaNs !

- Here you can use == to catch NaNs !

```
if(x == y) {
  // x == x will fail for NaNs
}
```

- or use `std::isnan(x)`

```
if(std::isnan(x)) {
  // x == x will fail for NaNs
}
```

- Also can use
  - **`std::isfinite(x)`** – returns true if x is a finite number (Normal numbers, subnormal numbers, +0, -0)
  - **`std::isinf(x)`** – returns true if x is positive or negative infinity, excludes NaN
  - **`std::isnormal(x)`** – true if x is a normal floating-point number, excludes: +/-0, subnormals, inf, NaN

READING

- **David Goldberg**: *What every computer scientist should know about floating-point arithmetic*. ACM Computing Surveys March. 23(1), pp. 5–48 (1991) https://doi.org/10.1145/103162.103163
- Tobias Weinzierl. Book Chapters 4, 6 (and 8 is extra)

- https://www.youtube.com/watch?v=wbxSTxhTmrs
- Numerical Computation Guide – Chapter 2: https://docs.oracle.com/cd/E19957-01/806-3568/ncg_math.html